

调用图上的规约生成与证明生成三条宏观路线

双访问序列、Bug 传播与最坏工程量

QCIP 技术调研

2026 年 9 月

摘要

本文只研究 annotation 驱动验证在调用图上的宏观调度问题。调用图节点统一表示函数或循环；每个节点有两次主访问：规约访问 $S(v)$ 和证明访问 $P(v)$ 。本文形式化三条路线：(I) 自顶向下、先在假设 callee 合同成立时完成 caller 证明，再递归证明 callee；(II) 自底向上、先生成并证明 callee，再生成并证明 caller；(III) 先自顶向下生成全图规约，再自底向上完成证明。顶层规约既可能外部给定，也可能由路线生成。本文还定义作用于节点的 $F(v)$ 操作：spec bug 沿反向调用边向上传播，proof bug 沿正向调用边向下定位，直到传播条件消失或到达叶节点。最后给出链、分叉、菱形共享、递归强连通分量和包含循环的调用图序列，并分析节点化实现与按路径重复实现的最坏工程量。

1 统一模型

1.1 扩展调用图

令扩展调用图为 $G = (V, E)$ ，其中 $V = V_F \uplus V_L$ ： V_F 是函数节点， V_L 是循环节点。边 $(u, v) \in E$ 表示 u 的验证依赖 v 的合同：

- 若 $u, v \in V_F$ ，它是普通函数调用；
- 若 $v \in V_L$ ，它表示函数或外层循环进入该循环；
- 嵌套循环用 $L_1 \rightarrow L_2$ 表示；
- 循环不再被单独抽象成特殊阶段，而是与 callee 一样拥有规约和证明操作。

循环节点 L 的规约就是 invariant、frame、退出关系等组成的合同 C_L ；其证明 $P(L)$ 包含初始化、保持性和退出三个义务。因此后续所有定义同时适用于函数和循环。

1.2 每个节点的两主访问

每个节点 $v \in V$ 有两个版本化操作：

$S(v)$ ： 生成、接收或固定合同 C_v ，

$P(v)$ ： $\text{Assume}(\{C_w \mid (v, w) \in E\}) \vdash \{\text{Pre}_v\} \text{Body}_v \{\text{Post}_v\}$.

这里“证明 v ”允许模块化地假设直接 callee 的合同成立；但整个验证完成前，每个被依赖合同都必须最终被证明或被明确标记为可信外部边界。Dafny 一类模块化验证器正是逐方法验证，并只使用其他方法的规格 [1]。

对 root r ，有两种输入模式：

- (a) **root spec 给定**： $S(r) = \text{Fix}(C_r^{\text{user}})$ ，不计算新语义；
- (b) **root spec 未给定**：路线必须生成 C_r 。此时仍需外部意图、可观察行为模板或“推断实现摘要”的策略，否则正确的功能规约并不唯一。

1.3 有效完成条件

设 T 是包含 $S(v)$ 、 $P(v)$ 和可选 $F(v)$ 的事件序列。一次 run 完成需要：

$$\forall v \in V_{\text{reachable}}. \quad S(v) \text{ 已固定} \wedge P(v) \text{ 已通过},$$

并且每个证明绑定的 callee spec version 与最终固定版本一致。若 C_v 变化，则所有依赖旧版本的证明不能继续作为当前结论。

2 三条宏观路线

2.1 路线 I：自顶向下，caller proof 先于 callee proof

该路线先确定当前节点规约，然后在假设直接 callee 合同成立的情况下完整证明当前节点；当前节点证明完成后，才递归证明刚才假设的 callee 合同。

对节点 v 的递归过程定义为：

- I1. 若 C_v 未给定或未固定，执行 $S(v)$ ；
- I2. 分析 Body_v 的调用点，为每个直接 callee w 生成或固定 C_w ；
- I3. 在假设所有 C_w 成立的条件下，完成 $P(v)$ ；
- I4. 按选定次序对尚未证明的 w 递归执行本过程；
- I5. 所有 callee 均通过后， $P(v)$ 才从“条件成立”升级为全局 accepted。

抽象序列为

$$S(v), S(w_1), \dots, S(w_k), P(v \mid C_{w_1}, \dots, C_{w_k}), T_I(w_1), \dots, T_I(w_k).$$

这条路线允许先验证顶层设计是否在假定接口下闭合，适合 caller 驱动的设计。但 callee 证明较晚：一旦 C_w 不可实现或必须修改，已经完成的 caller proof 可能全部失效。模块化逻辑允许这种证明顺序；工程风险来自合同版本失效，而不是逻辑规则本身。

root spec 两种情况。 若给定 C_r ，第一步只是登记并冻结用户规格；若未给定，则先从 root 需求生成 C_r 。其余遍历相同。

2.2 路线 II: 自底向上, callee spec/proof 先于 caller

该路线先递归处理所有 callee; 当前节点只能使用已经证明的 callee 合同来生成自己的合同并完成证明。

- II1. 对每个直接 callee w , 先递归完成 $T_{II}(w)$;
- II2. 基于 Body_v 、已证明的 C_w 和可选外部需求生成或确认 C_v ;
- II3. 使用已证明的 callee 合同完成 $P(v)$ 。

抽象序列为

$$T_{II}(w_1), \dots, T_{II}(w_k), S(v \mid C_{w_1}, \dots, C_{w_k}), P(v).$$

该路线与组合式 bottom-up summary analysis 最接近; bi-abduction 工作会独立分析过程并组合被调过程摘要 [2]。

root spec 未给定。 从叶节点开始根据实现生成并证明摘要, 逐层上推, 最后得到并证明 C_r 。

root spec 给定。 C_r^{user} 在 run 开始时登记为最终约束, 但不用于向下派生 callee 合同; 先完成底层合同, 最后检查实现摘要是否足以满足 C_r^{user} 。其优点是底层结论先稳定, 缺点是顶层不匹配可能很晚才暴露。

2.3 路线 III: spec 自顶向下, proof 自底向上

该路线明确分成两次全图遍历。

第一遍: Spec pass。 从 root 出发, 根据 caller 规约和调用点需求生成 callee 规约:

$$S(r) \prec S(v) \text{ for every reachable } r \rightsquigarrow v.$$

对共享 callee, 必须先收集多个 caller 的需求, 再固定一次稳定合同; 不能因第一次到达就忽略后续入边。

第二遍: Proof pass。 按逆依赖方向证明:

$$(v, w) \in E \implies P(w) \prec P(v).$$

所以整体是

$$\underbrace{S \text{ 的拓扑序}}_{\text{需求向下}} ; \underbrace{P \text{ 的逆拓扑序}}_{\text{证明向上}}.$$

给定模块接口后推断内部过程合同和循环 annotation 的 intra-module inference 与该路线的 spec pass 思想相近 [3]; 证明阶段仍采用模块化 callee-first。

该路线在进入任何 caller proof 前先固定可达规约, 并在 caller proof 前先验证 callee, 因此减少“先完成 caller proof、后发现 callee 合同不可实现”的重做。但它可能在尚无证明反馈时生成较多规约。

3 典型调用图的遍历序列

以下忽略同层节点之间无依赖时的可交换顺序。 S_X 表示 $S(X)$, P_X 表示 $P(X)$ 。若 root spec 给定, 把 S_A 理解为 $\text{Fix}(C_A^{\text{user}})$; 若未给定, 则理解为生成 C_A 。

3.1 链: $A \rightarrow B \rightarrow C$

路线	一个合法序列
I	$S_A, S_B, P_A, S_C, P_B, P_C$
II	$S_C, P_C, S_B, P_B, S_A, P_A$
III	$S_A, S_B, S_C, P_C, P_B, P_A$

路线 I 中, P_A 假设 C_B ; 证明 B 时才生成 C_C 。路线 II 的每个节点在后序遍历中连续执行 spec/proof。路线 III 是严格的前序 spec、后序 proof。

3.2 分叉: $A \rightarrow B$ 且 $A \rightarrow C$

路线	一个合法序列
I	$S_A, S_B, S_C, P_A, P_B, P_C$
II	$S_B, P_B, S_C, P_C, S_A, P_A$
III	$S_A, S_B, S_C, P_B, P_C, P_A$

其中 B, C 可并行; 路线 I 只有在 P_B, P_C 都通过后, P_A 才最终解除假设。

3.3 菱形共享: $A \rightarrow B, C, B \rightarrow D, C \rightarrow D$

路线	一个合法序列
I	$S_A, S_B, S_C, P_A, S_D, P_B, P_D, P_C$
II	$S_D, P_D, S_B, P_B, S_C, P_C, S_A, P_A$
III	$S_A, S_B, S_C, S_D, P_D, P_B, P_C, P_A$

该图揭示三条路线的关键差异:

- 路线 I 可能在证明 B 时根据第一个调用上下文固定 C_D ; 之后证明 C 才发现第二个需求, 因而必须加强或增加 D 的派生规约。
- 路线 II 先从 D 实现得到稳定能力, 再供 B, C 使用, 但这个能力可能不够满足顶层意图。
- 路线 III 在 spec pass 中汇总 $B \rightarrow D$ 与 $C \rightarrow D$ 两条入边的需求后只固定一次 C_D , 随后从 D 向上证明。

3.4 递归: $A \leftrightarrow B$

有环图不存在普通拓扑序。先将 G 缩点为 SCC DAG。对于 $K = \{A, B\} = \text{SCC}(A)$, 必须联合固定合同族 $C_K = \{C_A, C_B\}$, 并证明

$$\text{Assume}(C_A, C_B) \vdash \{\text{Pre}_A\}\text{Body}_A\{\text{Post}_A\} \wedge \{\text{Pre}_B\}\text{Body}_B\{\text{Post}_B\}.$$

三条路线在 SCC 内退化为:

路线	SCC 内处理
I	S_A, S_B, P_A, P_B , 两个 proof 完成后联合提交
II	无真正叶节点; 必须先求 C_A, C_B 的联合不动点, 再联合证明
III	S_A, S_B 联合冻结, 然后 P_A, P_B 联合提交

SCC 之间仍可分别应用三条宏观路线。也就是说, “任意 CG 的遍历序列” 应定义在缩点 DAG 上, SCC 内使用联合事件块。

3.5 循环作为 callee

设函数 A 包含外层循环 L_1 , L_1 调用函数 B 并包含内层循环 L_2 :

$$A \rightarrow L_1, \quad L_1 \rightarrow B, \quad L_1 \rightarrow L_2.$$

则合法序列为:

路线	一个合法序列
I	$S_A, S_{L_1}, P_A, S_B, S_{L_2}, P_{L_1}, P_B, P_{L_2}$
II	$S_B, P_B, S_{L_2}, P_{L_2}, S_{L_1}, P_{L_1}, S_A, P_A$
III	$S_A, S_{L_1}, S_B, S_{L_2}, P_B, P_{L_2}, P_{L_1}, P_A$

这里 S_{L_i} 就是生成 invariant/loop contract, P_{L_i} 就是证明初始化、保持和退出, 不需要第四条 “loop 专用路线”。

4 FindBug 操作与传播方向

4.1 节点化 Bug 操作

$F(v)$ 作用于一个节点及其当前 spec/proof version。返回值至少区分:

$$F(v) \in \{\text{no bug}, \text{spec bug}, \text{proof bug}, \text{implementation bug}, \text{unknown}\}.$$

本文按用户指定的两类传播规则建模。

4.2 Spec bug 向上传播

若 C_v 错误、过弱、过强或与调用点需求不一致，则受影响集合位于反向可达闭包：

$$U(v) = \{u \mid u \rightsquigarrow v\} \cup \{v\} = \text{Anc}(v) \cup \{v\}.$$

传播不是无条件覆盖所有祖先。定义依赖谓词

$$\text{Uses}(u, v, \delta C_v),$$

表示 u 的 spec/proof 实际使用了本次变化的合同分量。只沿满足该谓词的反向边传播：

$$(u, v) \in E \wedge \text{Uses}(u, v, \delta C_v) \implies \text{stale}(u).$$

当某个 caller 不使用变化部分、存在不变的派生规约适配层，或到达 root 时停止。

4.3 Proof bug 向下传播

若 $P(v)$ 失败且当前尚不能断定是 v 本地实现错误，则沿 callee/loop 边向下查找：

$$D(v) = \{w \mid v \rightsquigarrow w\} \cup \{v\} = \text{Desc}(v) \cup \{v\}.$$

定义 $\text{Delegates}(v, w, o)$ 表示未闭合同务 o 依赖 w 的合同或证明。仅当该谓词成立时继续传播到 w 。停止条件为：

1. 在当前节点已定位为本地代码/证明脚本错误；
2. callee 的 accepted proof 与当前版本匹配，且义务不依赖其未证明性质；
3. 到达叶节点；
4. 发现真正问题是 spec bug，随后切换为向上传播。

4.4 版本失效原则

每个结果绑定

$$\text{version}(v) = H(C_v, \text{Body}_v, \{\text{version}(w) : (v, w) \in E\}).$$

spec bug 修改 C_v 后，旧 $P(v)$ 和所有使用旧版本的 caller proof 自动 stale。节点版本和边依赖必须去重；否则同一个共享 callee 会沿多条路径被重复传播。

5 工程量模型

5.1 无返工时的基线

令 $n = |V|$ 、 $m = |E|$ ，并令 s_v, p_v 分别为生成规约和证明节点的实际成本， q_e 为检查调用边兼容性的成本。一次无返工 run 的工程量为

$$W_0 = \sum_{v \in V} (s_v + p_v) + \sum_{e \in E} q_e.$$

若把各操作视为单位成本，则三条路线都是 $W_0 = \Theta(n + m)$ 。路线差异不在首次成功遍历的渐近阶，而在：何时暴露矛盾、多少已完成结果会因 bug 失效、需要保存多大的假设前沿，以及共享节点是否重复处理。

root spec 给定只省去或显著降低一个 s_r ，不会改变渐近阶；但它能约束搜索空间，并改变 bug 被发现的时间。

5.2 一次 Bug 传播的最坏成本

对 spec bug v ，节点化向上传播成本上界为

$$W_S(v) = O(|U(v)| + |E[U(v)]|),$$

加上这些节点中已完成 spec/proof 的重做成本。对 proof bug，向下搜索上界为

$$W_P(v) = O(|D(v)| + |E[D(v)]|).$$

最坏情况下 $U(v) = V$ 或 $D(v) = V$ ，所以单次 bug wave 为 $O(n + m)$ 。

5.3 多轮返工的通用上界

若共有 b 个相互独立的 bug/revision wave，并且每轮都可能覆盖全图，则调度和重验工程量为

$$W = O((b + 1)(n + m))$$

(未计 solver 内部复杂度)。若 $b = \Theta(n)$ ，则 $W = O(n(n + m))$ 。稀疏 CG ($m = O(n)$) 为 $O(n^2)$ ；稠密 CG ($m = \Theta(n^2)$) 为 $O(n^3)$ 。

5.4 三条路线各自的最坏返工形态

路线 I。 caller proof 先于 callee proof。链长为 n 时，最深 callee 的 spec/proof 问题可能使此前 $n - 1$ 个 caller proof 失效；若每深入一层都发生一次需要向上修正的问题，重做量为

$$1 + 2 + \cdots + n = \Theta(n^2).$$

一般图上， b 轮最坏为 $O(b(n + m))$ 。路线 I 的主要风险是 *speculative proof debt*：已经完成但仍依赖未验证合同的 caller proof 数量。

路线 II。 callee proof 先完成，因此不会因尚未验证的 callee 直接废弃已完成 caller proof。在 root spec 未给定、摘要推断单调且一次收敛时，最接近 $\Theta(n + m)$ 。但若 root spec 给定且直到最后才发现底层摘要不足，则一次顶层不匹配可触发全图向下加强； b 次加强仍为 $O(b(n + m))$ 。此外，自底向上生成的路径敏感摘要可能随分支组合指数增长，这属于合同表示大小而非遍历次数。

路线 III。 先固定规约，再从叶到根证明。callee proof bug 会在对应 caller proof 尚未开始时暴露，因此通常最少浪费已完成 proof；但如果证明失败要求修改 callee spec，spec bug 仍会向上使祖先规约 stale。 b 轮最坏同样是 $O(b(n + m))$ ，只是常数项通常更偏向重新生成 spec，而不是重新做已经完成的 caller proof。

5.5 按路径传播会导致指数爆炸

上述多项式上界依赖“节点访问去重 + 版本化边依赖”。若实现按调用路径而不是节点闭包传播，具有 k 层二选一菱形的 DAG 可以拥有 2^k 条 root-to-leaf 路径，而节点数只有 $O(k)$ 。此时同一 bug 会被重复报告/重验

$$\Theta(2^k) = \Theta(2^{n/2})$$

次。工程实现必须：

- 以 $(v, \text{version})$ 为访问键；
- 对共享 callee 汇总入边需求；
- 每个 bug wave 对一个节点最多入队一次；
- 在 SCC 层面而不是递归路径层面提交结果。

5.6 SCC 与候选规约搜索

若 SCC K 含 r 个节点和 e_K 条内部边，一次固定候选合同后的联合检查仍是 $O(r + e_K)$ 个调度操作；但找到合同不动点可能需要 h 轮：

$$O(h(r + e_K)).$$

若每个节点有 k 个互不约束的合同候选，朴素联合枚举可达 k^r 。Houdini 式候选删减通过反复调用 checker 删除不可证候选 [4]；ICE/CEGIS 则通过反例迭代学习不变量 [5]。它们改变候选搜索策略，但不消除一般程序验证的不可判定性。因此必须区分：

1. **CG 调度工程量**：在正确去重下按 wave 为 $O(n + m)$ ；
2. **annotation 搜索工程量**：可能是多项式、指数或不收敛；
3. **单个 VC 的证明复杂度**：取决于逻辑和 solver，一般不存在统一有限最坏上界。

6 路线比较与工程选择

路线	优点	主要风险	更适合的场景
I: caller proof 先行	很早验证顶层设计在假定接口下是否闭合；caller 驱动接口直接	callee 合同不可实现时，祖先 proof 大面积失效	root 需求最重要、callee 合同可快速稳定或来自可信团队
II: bottom-up	每个 caller 只依赖已证明 callee；底层复用和增量性好	给定 root 规格时可能很晚才发现实现摘要不足；实现导出的 spec 未必表达意图	底层库先建设、历史代码摘要推断、root spec 未给定

路线	优点	主要风险	更适合的场景
III: spec-down/proof-up	在 proof 前统一处理需求; callee 失败发生在 caller proof 前	spec pass 可能生成之后证明不需要或不可实现的合同; 仍需回流	高可信模块化验证、明确 root 需求、共享 callee 较多

三条路线都可以合法; 它们的核心差异是两个偏序:

$$\prec_S \text{ (spec 顺序)}, \quad \prec_P \text{ (proof 顺序)}.$$

路线 I 允许 caller 的 P 先于 callee; 路线 II 的 S 和 P 都是 bottom-up; 路线 III 的 S top-down、P bottom-up。

7 对 QCP 编排的直接含义

若 QCP 要显式支持三条路线, controller 不应只记录 phase, 还应为每个节点记录

(spec status, spec version, proof status, proof dependencies, bug state).

并提供:

1. CG/SCC 构造和稳定节点 ID;
2. S/P 双访问序列生成器;
3. 路线 I 的“条件完成 proof”状态, 直到 callee proof 全部 discharge;
4. 路线 II 的 bottom-up summary/spec 接口;
5. 路线 III 的 top-down demand aggregation 和 reverse proof schedule;
6. spec bug 的 reverse-reachability invalidation;
7. proof bug 的 forward-reachability diagnostic search;
8. 版本化、节点去重和 SCC 联合提交, 避免按路径指数重复。

现有 QCP 的单 case 流程更接近路线 III 的局部版本: 先 annotation, 再生成 VC, 最后 proving; 但只有把函数与循环组织成上述 CG 节点、明确 S/P 两次访问及 bug 传播, 才能讨论跨函数的三条完整宏观路线。

8 结论

对任意调用图, 先缩并 SCC 得到 DAG, 再选择三种调度之一:

- I spec-down, caller proof 可先行, callee proof 延后 discharge;
- II callee spec/proof 全部 bottom-up;
- III spec top-down, proof bottom-up。

每个 function/loop 节点恰有一次当前版本的 spec 主访问和一次 proof 主访问；修复产生新版本和新 wave，而不是把同一旧版本沿每条路径重复遍历。无 bug 时三条路线的 CG 调度均为 $\Theta(n + m)$ 。单个全图传播 bug 为 $O(n + m)$ ； b 轮为 $O(b(n + m))$ ；当 $b = \Theta(n)$ 时，稀疏图最坏 $O(n^2)$ ，稠密图最坏 $O(n^3)$ 。若不按节点去重而按路径传播，则即使 DAG 也可能指数爆炸。annotation 候选搜索和 VC 求解还有独立的指数或不可判定复杂度，不能与调用图遍历成本混为一谈。

参考文献

- [1] K. Rustan M. Leino. Dafny: An Automatic Program Verifier for Functional Correctness. In *LPAR-16*, LNCS 6355, pp. 348–370, 2010. DOI: 10.1007/978-3-642-17511-4_20.
- [2] Cristiano Calcagno, Dino Distefano, Peter W. O’Hearn, and Hongseok Yang. Compositional Shape Analysis by Means of Bi-Abduction. In *POPL 2009*, pp. 289–300, 2009. DOI: 10.1145/1480881.1480917.
- [3] Shuvendu K. Lahiri, Shaz Qadeer, Juan P. Galeotti, Jan W. Voung, and Thomas Wies. Intra-Module Inference. In *CAV 2009*, LNCS 5643, pp. 493–508, 2009. DOI: 10.1007/978-3-642-02658-4_37.
- [4] Cormac Flanagan and K. Rustan M. Leino. Houdini, an Annotation Assistant for ESC/Java. In *FME 2001*, LNCS 2021, pp. 500–517, 2001. DOI: 10.1007/3-540-45251-6_29.
- [5] Pranav Garg, Christof Löding, P. Madhusudan, and Daniel Neider. ICE: A Robust Framework for Learning Invariants. In *CAV 2014*, LNCS 8559, pp. 69–87, 2014. DOI: 10.1007/978-3-319-08867-9_5.